

Glosario - Semana 06

A

AbortController

Interfaz de la Web API que permite cancelar peticiones fetch y otras operaciones asíncronas. Esencial para evitar memory leaks en hooks de fetching.

```
const controller = new AbortController();
fetch(url, { signal: controller.signal });
controller.abort(); // Cancela la petición
```

B

Barrel Export

Patrón de exportación que consolida múltiples exports en un solo archivo `index.ts`, simplificando las importaciones.

```
// hooks/index.ts
export { useToggle } from './useToggle';
export { useFetch } from './useFetch';
```

C

Cleanup Function

Función retornada por `useEffect` que se ejecuta cuando el componente se desmonta o antes de re-ejecutar el efecto. Esencial para limpiar timers, suscripciones y cancelar requests.

```
useEffect(() => {
  const timer = setInterval(tick, 1000);
  return () => clearInterval(timer); // Cleanup
}, []);
```

Composición de Hooks

Patrón de crear hooks que utilizan otros hooks internamente, permitiendo abstracciones de alto nivel.

```
const useAuth = () => {
  const { value: user } = useLocalStorage('user', null);
  const { execute: login } = useFetch();
  // Combina múltiples hooks
};
```

Custom Hook

Función de JavaScript/TypeScript que comienza con "use" y puede llamar a otros hooks. Permite extraer y reutilizar lógica con estado.

D

Debounce

Técnica de optimización que retrasa la ejecución de una función hasta que pase un tiempo determinado sin nuevas invocaciones.

```
const debouncedValue = useDebounce(searchTerm, 300);
```

Dependency Array

Array de dependencias pasado a `useEffect`, `useCallback`, o `useMemo` que determina cuándo re-ejecutar el hook.

E

ESM (ES Modules)

Sistema de módulos estándar de JavaScript usado por Vite, que permite `import/export` nativos.

G

Generics

Feature de TypeScript que permite crear componentes y funciones reutilizables que trabajan con cualquier tipo.

```
const useLocalStorage = <T,>(key: string, initial: T): T => { ... };
```

H

HMR (Hot Module Replacement)

Característica de Vite que actualiza módulos en el navegador sin recargar toda la página, preservando el estado de la aplicación.

Hook Rules

Reglas que deben seguirse al usar hooks:

1. Solo llamar hooks en el nivel superior
2. Solo llamar hooks desde componentes funcionales o custom hooks

L

Lazy Initialization

Técnica donde `useState` recibe una función en lugar de un valor, ejecutándose solo en el primer render.

```
const [state, setState] = useState(() => {  
  return expensiveComputation();  
});
```

M

Memoization

Técnica de optimización que cachea resultados de funciones para evitar recálculos innecesarios. Implementada con `useMemo` y `useCallback`.

R

Ref

Objeto mutable ({ current: T }) que persiste durante todo el ciclo de vida del componente sin causar re-renders.

```
const timerRef = useRef<NodeJS.Timeout | null>(null);
```

S

Signal (AbortSignal)

Objeto que permite comunicar cuando una operación debe ser abortada. Creado a partir de `AbortController`.

State Derivado

Estado que se calcula a partir de otro estado existente, en lugar de almacenarse por separado.

```
const isAuthenticated = user !== null && token !== null;
```

T

Type Inference

Capacidad de TypeScript de deducir tipos automáticamente sin anotaciones explícitas.

```
const count = useState(0); // TypeScript infiere number
```

U

useCallback

Hook que memoriza una función, retornando la misma referencia entre renders si las dependencias no cambian.

useEffect

Hook para ejecutar efectos secundarios (fetching, suscripciones, DOM manual) después del render.

useMemo

Hook que memoriza un valor calculado, recalculándolo solo cuando las dependencias cambian.

useRef

Hook que retorna un objeto ref mutable que persiste durante el ciclo de vida del componente.

useState

Hook para agregar estado local a componentes funcionales.

V

Vite

Build tool moderno para desarrollo web que usa ESM nativo para desarrollo rápido y Rollup para producción optimizada.

```
pnpm create vite@latest
```